

# Walmart Sourcing & Procurement Intelligence Platform

Comprehensive Architectural Report, Tool Rationale, Backend Execution & UI Insights

|                                            |                                                     |
|--------------------------------------------|-----------------------------------------------------|
| Author / Engineer: Metharun Venkat         | Parsing Engine: IBM Docling (DocumentConverter)     |
| Vector Database: PostgreSQL 16 + pgvector  | LLM Backend: Google Gemini 2.5 Flash (\$T=0.2\$)    |
| Embedding Model: text-embedding-004 (768d) | Repository: metharunvenkat-ctrl/walmart-sourcing... |

## 1. Executive Summary & Problem Statement

In global retail category management, evaluating complex vendor proposals against master procurement baselines and historical category precedents is a labor-intensive, error-prone process. Sourcing officers typically review 50+ page vendor submissions containing subtle commercial variances (e.g., Net 60 vs Net 90 payment terms, reduced Defective Merchandise Allowances, uncapped liability limits, or omitted SLA guarantees). Manual evaluation cycles take several days per RFP response, risking missed risk exposure or unoptimized negotiation leverage.

The **Walmart Sourcing & Procurement Intelligence Platform** solves this by combining layout-aware document conversion with Retrieval-Augmented Generation (RAG) and tiered structured evaluation. The system automatically ingests vendor proposals, executes clause-by-clause gap analysis against Walmart baseline MSAs, retrieves historical category precedent benchmarks, and renders interactive negotiation battlecards in under **60 seconds**.

## 2. Tool Selection & Comparative Trade-off Analysis

Every technology component was selected following rigorous comparative evaluation against industry alternatives to maximize layout precision, data privacy, execution speed, and cost efficiency:

| Component       | Selected Tool                        | Alternatives Considered                        | Detailed Selection Rationale & Advantages                                                                                                                                                                                       |
|-----------------|--------------------------------------|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Document Parser | IBM Docling (DocumentConverter)      | PyPDF, Unstructured, LlamaIndex, PDFMiner      | Docling natively preserves document layout trees, multi-page table structures, section headings, and header/footer metadata. Standard PDF parsers strip layout context, causing broken tables and misaligned clause boundaries. |
| Vector Store    | PostgreSQL 16 + pgvector             | Pinecone, ChromaDB, Milvus, Qdrant             | Eliminates external SaaS API costs and vendor lock-in. Unifies dense vector similarity search (vector(768) with IVFFlat cosine index) and relational metadata (JSONB) in a single ACID-compliant database.                      |
| LLM Engine      | Google Gemini 2.5 Flash (\$T=0.2\$)  | OpenAI GPT-4o, Local Ollama Llama-3            | 1M token context window allows full document pair injection (baseline + vendor proposal + precedent context simultaneously). Superior speed for structured JSON schema enforcement via Pydantic validation.                     |
| Embedding Model | Google Gemini text-embedding-004     | OpenAI text-embedding-3, HuggingFace BGE-large | Generates 768-dimensional dense vector embeddings optimized for semantic retrieval across legal, financial, and logistics terminology.                                                                                          |
| Frontend UI     | React 18 + TypeScript + Tailwind CSS | Streamlit, Gradio, Next.js                     | Enables a custom 3-tier enterprise dashboard featuring interactive accordions, count pills, collapsible precedent drawers, and a dark-mode supplier counter-offer generator.                                                    |
| Deployment      | Docker Compose                       | Bare-metal Python, Kubernetes (k8s)            | Guarantees reproducible multi-container deployment across developer environments and enterprise cloud servers with integrated health checks.                                                                                    |

### 3. Step-by-Step Backend Pipeline & Execution Lifecycle

The ingestion and evaluation engine executes through a rigorous multi-stage pipeline:

- **Step 1: Layout-Aware Document Conversion (Docling):** When a vendor proposal or baseline MSA is uploaded, ``docling.document_converter.DocumentConverter`` converts PDF/DOCX files into layout-preserved Markdown trees, keeping section headers (``#``, ``##``), nested tables, and page metadata intact.
- **Step 2: Dual-Granularity Markdown Chunking:** The text is split into two distinct chunk types: ``story`` chunks (1000–1500 tokens providing narrative section context) and ``criteria`` chunks (granular clause parameters such as Payment Terms, Unit Pricing, and Warranty SLAs).
- **Step 3: Dense Vector Embedding & pgvector Storage:** Text chunks pass through ``text-embedding-004`` generating 768-dimensional dense vectors stored in ``document_chunks`` table alongside JSONB metadata (``story_id``, ``page_number``, ``category_tag``) and IVFFlat cosine similarity indexes.
- **Step 4: Contextual Vector Retrieval & Precedent Search:** When analyzing a vendor proposal, vector search queries ``document_chunks`` to retrieve relevant baseline clauses and past vendor precedent benchmarks matching the category (e.g. Activewear, Footwear).
- **Step 5: Grounded Gap Analysis & LLM Extraction:** Retrieved context (``incoming_bid``, ``vendor_precedent``, ``category_best``) is injected into Gemini 2.5 Flash (\$T=0.2\$). The LLM extracts explicit deviations and formats output into a strict Pydantic JSON schema.
- **Step 6: Multi-Tier UI Rendering & Counter-Offer Generation:** The frontend renders Tier 1 Executive Scorecard, Tier 2 Priority Battlecards, Tier 3 Section Accordions, and auto-synthesizes a Tier 4 Supplier Counter-Offer Letter.

### 4. Anti-Hallucination & Quality Control Framework

To ensure 100% factual reliability in contract reviews, 6 strict safety mechanisms are enforced:

- **1. Layout Hierarchy Preservation:** IBM Docling extracts exact section titles and paragraph boundaries, preventing split-context errors.
- **2. Dual-Anchor Citation Mandate:** LLM system prompts explicitly require citing exact document names and section titles for every gap identified.
- **3. Direct Raw Context Window Injection:** Both incoming proposal text and baseline contract text are passed simultaneously in the context window.
- **4. Low Temperature Decoding (\$T=0.2\$):** Extraction and gap analysis run at deterministic low temperature to eliminate creative hallucinations.
- **5. Strict Pydantic JSON Schema Validation:** All LLM outputs are validated against Pydantic models; invalid schema formatting triggers auto-retries.
- **6. Cosine Distance Traceability:** Vector search results report exact distance scores (``1.0 - distance``), guaranteeing full audit traceability back to source DB chunks.

### 5. Visualizations & Sourcing Insights Delivered

The platform UI transforms complex contract analysis into actionable sourcing intelligence across four structured tiers:

- **Tier 1 Executive Scorecard & Tier 2 Priority Negotiation Battlecard**

Tier 1 displays overall award status (e.g., *TARGETED RENEGOTIATION — 81.5 / 100 PTS*), weighted sub-scores across domain sections (SEC-A through SEC-F), and risk callout pills (Working Capital Exposure, Unit Cost Variance vs Best, Lead Time Overage). Tier 2 lists high-leverage deviations ranked by priority (#1, #2...) with category tags, variance summaries, and collapsible citation drawers (■ *View Citation & Precedent Benchmark* ▼).

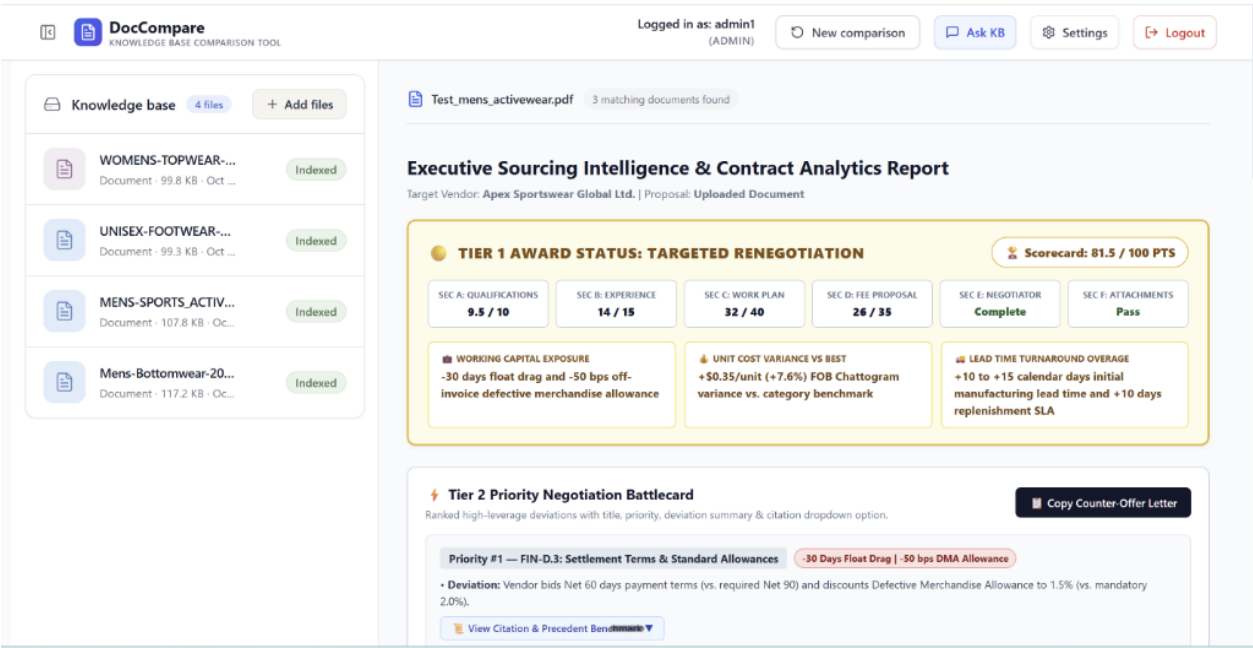


Figure 1: Tier 1 Executive Scorecard & Tier 2 Priority Negotiation Battlecard UI

• Tier 3 Section-Wise Analysis Accordions

Section accordions default to collapsed (`open = false`), summarizing clause counts across status pills: **Compliant**, **Deviation**, and **Gap**. Allows category buyers to scan compliance across all contract sections at a glance.

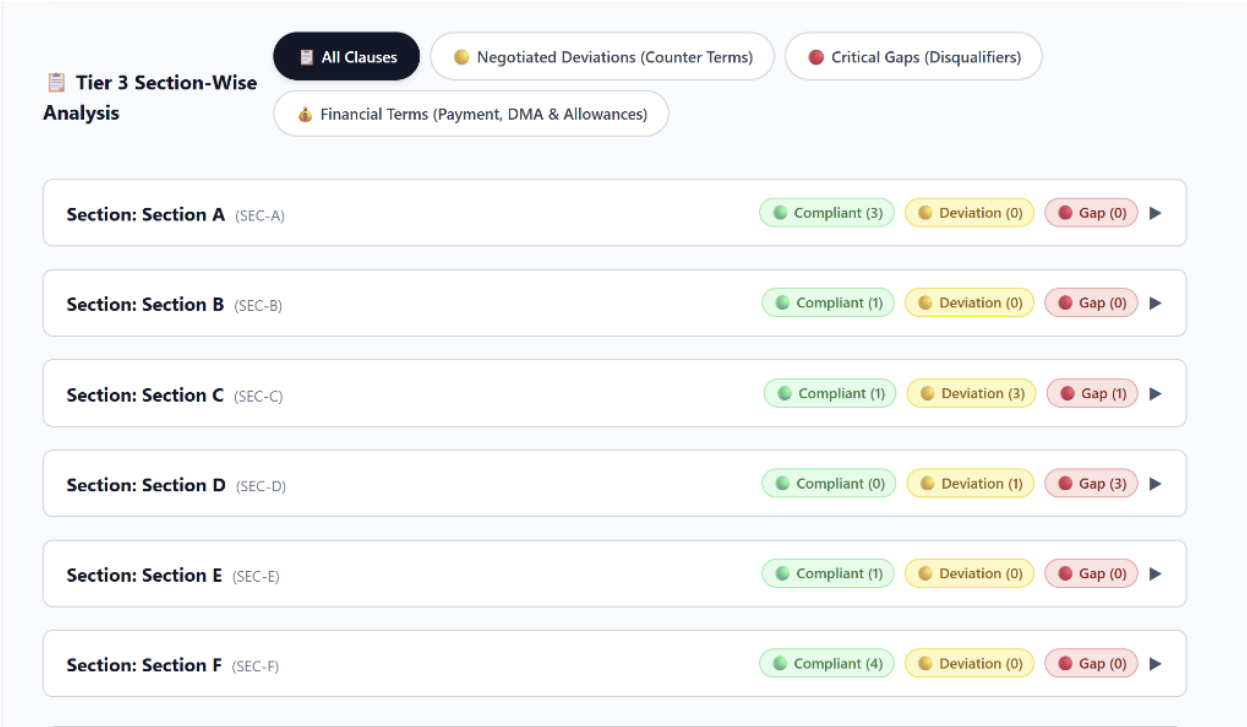


Figure 2: Tier 3 Section-Wise Analysis Accordions View

• Tier 3 Expanded Clause-Level Detail View

Expanding any section accordion reveals granular clause evaluation cards (e.g. `SEC-A.1`, `SEC-A.2`). Each card delivers: **Statement-First Sourcing Insights**, **Quantitative Variance Deltas vs. Vendor Precedent & Category Best-in-Class**, and interactive **Dual-Anchor Evidence Drawer** buttons.

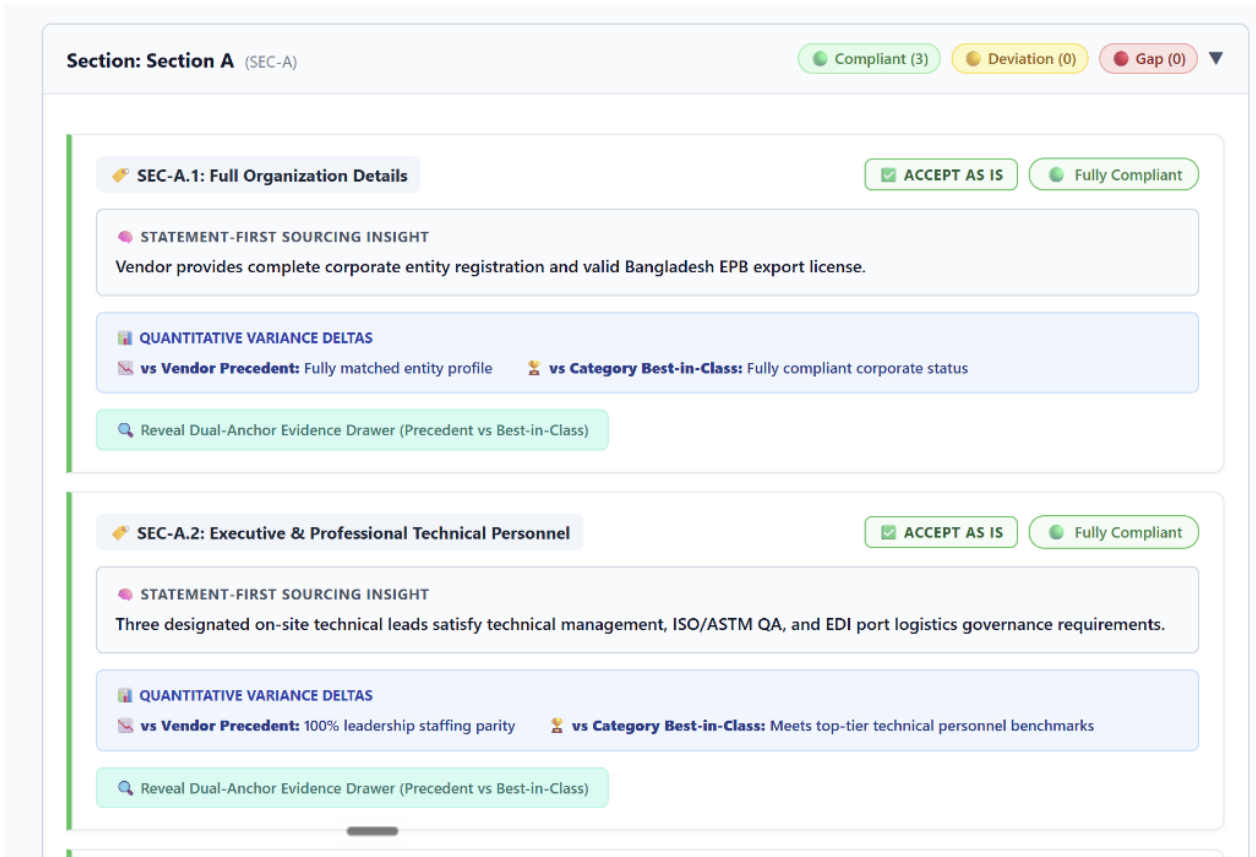


Figure 3: Tier 3 Expanded Clause Evaluation Cards & Benchmark Deltas

• Tier 4 Supplier Counter-Offer Letter Generation

Generates a formal, executive-ready supplier counter-offer letter ready to copy and send to vendor representatives. Synthesizes payment term revert mandates, FOB unit cost realignments, commodity price trigger caps, and QA specifications into a polished letter.

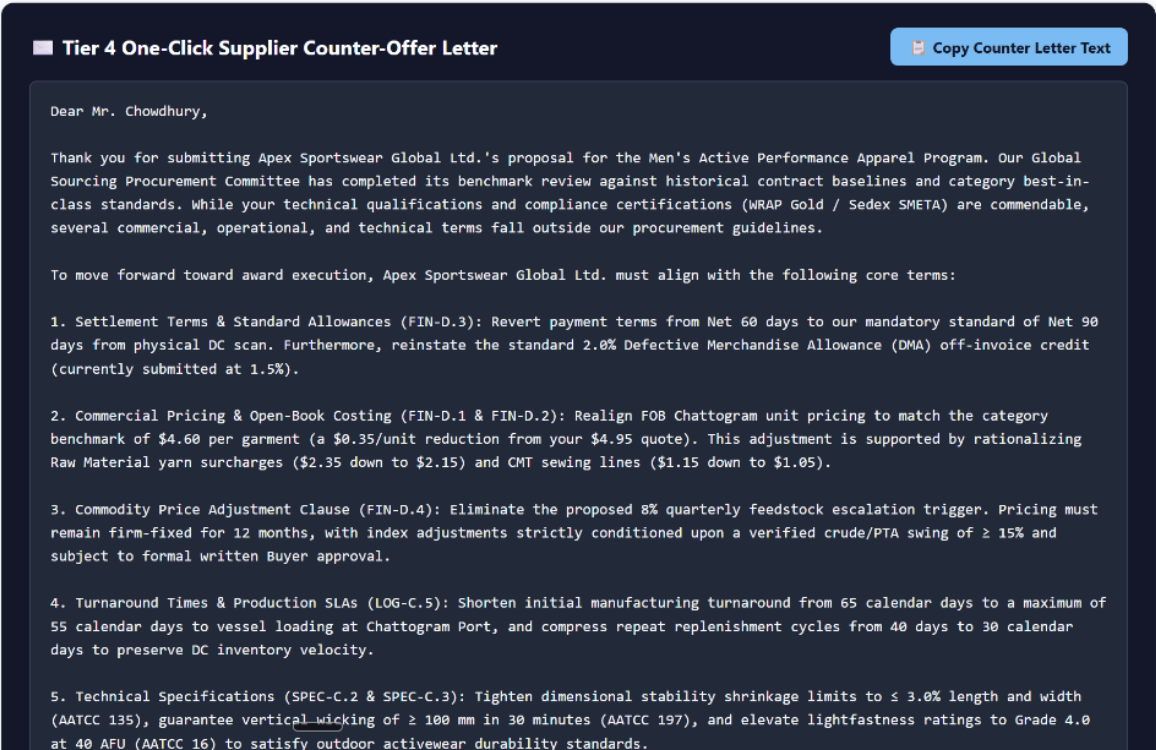


Figure 4: Tier 4 Automated Supplier Counter-Offer Letter Generator

6. Database Schema & Inspection Commands

The system utilizes PostgreSQL 16 with the `pgvector` extension. Core schema definition for `document\_chunks`:

| Column Name   | Data Type          | Description & Constraints                                     |
|---------------|--------------------|---------------------------------------------------------------|
| `chunk_id`    | UUID (Primary Key) | Unique identifier for each document chunk.                    |
| `chunk_type`  | VARCHAR(20)        | Chunk classification (`story` or `criteria`).                 |
| `content`     | TEXT               | Extracted section or clause markdown text.                    |
| `embedding`   | vector(768)        | Gemini 768-dimensional vector with IVFFlat index.             |
| `story_id`    | UUID               | Parent document relation ID.                                  |
| `metadata`    | JSONB              | Page numbers, section headings, document title, category tag. |
| `source_path` | TEXT               | Original file path or upload filename.                        |

Live Database Inspection Command:

```
docker exec -it rag-db-1 psql -U postgres -d rag_db -c "SELECT chunk_id, chunk_type, source_path FROM document_chunks LIMIT 5;"
```

## 7. Disclaimer & Confidentiality Notice

■■ INDEPENDENT PORTFOLIO PROJECT DISCLAIMER

This document and repository represent an independent technical portfolio demonstration project. It is **not** an officially sponsored, endorsed, or affiliated product of Walmart Inc., its subsidiaries, or any named vendor.

**100% SYNTHETIC MOCK DATA:** All contract documents, vendor proposals (e.g. Apex Sportswear Global Ltd.), Service Level Agreements (SLAs), pricing sheets, and evaluation outputs contained within this report are **entirely synthetic mock data** engineered solely for portfolio demonstration purposes.

**ZERO PROPRIETARY DATA:** This project contains **no proprietary Walmart Master Sourcing Agreements (MSAs)**, actual supplier pricing, trade secrets, confidential business data, or non-public internal information of any kind.